Compute Offload Framework for Cloud/Edge Media Rendering in Object-Based Media

Leo Wong (Student ID: 39435377)
Project Supervisor: Barry Porter

1 Introduction

Media services have become increasingly popular with growing volumes of video uploaded across various platforms. Alongside this trend, there is a rising demand for customisable media where viewers can select their preferred element combinations. However, traditional streamed media cannot efficiently deliver such personalised content due to the extreme storage required by all possible combinations. Object-based media (OBM) is an emerging paradigm which offers higher flexibility and customisability by separating media objects until delivery [ABC+14].

However, an OBM WebAssembly player requires considerable computational power on the viewer's device to composite and display the final video. To address this, an offloading framework is needed to reduce the computational pressure of the viewer's device by pre-rendering the requested combination into a video segment [PRR26]. In order to minimise the viewer perceived latency and maintain reasonable offload site utilisation, this project is proposed to investigate OBM-specific load-balancing strategies and offloading features that improve overall user experience under realistic workloads.

2 Background

2.1 OBM Offloading Pipeline & Request Heterogeneity

The OBM offload site is mainly responsible for rendering the client's requested combination into a 10-second video segment at 25 fps [PRR26]. The rendering pipeline consists of three stages, each with a distinct purpose and execution backend:

Table 1: Dana Offloading Pipeline Overview

Stage	Purpose	Device / Backend
1. Decoding	Decompressing incoming video assets and resources	CPU (libavcodec), GPU (VA-API)
2. Composition & Readback	Layer compositing, rasterization, and framebuffer extraction	CPU (SDL), CPU software rasterizer (Mesa llvmpipe), GPU (OpenGL)
3. Encoding	Final stream and frame compression	CPU, GPU

In addition to hardware differences across offload sites, the computational workload of each video request is highly dependent on the selected combination of layers. This dual heterogeneity of worker hardware and task combination creates a wide range in task completion times, even

before accounting for networking factors. On the other hand, networking factors like latency and bandwidth are also significant determinants of the system performance, driven by the multi-round inter-server communication and the vast volume of video segment and asset transfers constrained by link bandwidth.

For instance, rendering lightweight 2D overlays only requires under 50ms on a GPU, whereas full-frame chroma-keying on a CPU exceeds 5000ms.

2.2 Load-Balancing & Optimisation Complexity

Due to the multidimensional complexity of OBM offloading, traditional static load balancing (SLB) algorithms such as Round Robin, Randomisation and IP Hash are insufficient to capture the system loads. This limitation arises from the assumption of uniform task sizes and their lack of runtime feedback. Furthermore, simple dynamic load balancing (DLB) algorithms like Throttled, Equally Spread Current Execution and Minimum Execution Time, focus solely on a single system variable [SJA22, LGS+23, BSB+01], often producing unbalanced scheduling decisions that lead to worker underload or overload. To resolve the problem, we require DLB mechanisms capable of capturing multi-variables including worker makespan, processing power and networking factors.

Beyond load balancing, certain computational optimisations are needed to reduce repeated work and client perceived latency. However, regular caching is ineffective due to the excessive amount of combinations and large volume of video data. To address this, we propose caching the commonly shared computationally heavy partial segment, which preserving the flexibility of OBM with reasonable storage requirement. Especially given the irregular processing cost across combinations [DRP25]. Beside caching, other approaches such as proactive prediction and request coalescing represent promising optimisation directions [PRR26].

2.3 User Behaviors & Preference

Krishnan and Sitaraman [KS12] established a causal relationship between streaming latency and viewer abandonment. Crucially, their findings show that viewer patience begins to decline rapidly once startup delay exceeds 2 seconds. Beyond this 2-second threshold, every additional 1-second delay results in approximately 5.8% increase in viewer abandonment. Furthermore, playback interruptions and rebuffering severely degrade engagement; even a 1% normalized rebuffer delay causes viewers to watch 5.02% less of the video [KS12]. In an OBM offload environment, where compute-intensive rendering times can easily span several seconds, keeping client perceived latency strictly within this 2-second window is key to preventing immediate user abandonment.

Cha et al. [CKR+07] analysed user access distributions and demonstrated that video popularity follows extreme Pareto skewness (the 80–20 rule). Their measurements revealed that access patterns are even more concentrated than classic Pareto distributions; the top 10% to 20% of popular video items account for nearly 80% of aggregate viewer requests, while the remaining content receives very few views [CKR+07]. In the context of OBM, while combinatorial flexibility allows hundreds of unique layer permutations, actual viewer choices similarly cluster around a small set of popular default combinations.

3 The Proposed Project

3.1 Objectives

This project aims to investigate the optimisation space of an OBM offload rendering framework. In order to understand the interaction between different system variables, the following objectives

are defined:

- Develop a **distributed OBM system** incorporating various **load balancing algorithms** and **computational optimisations**.
- Conduct **real-world experiments** and **software simulations** to evaluate the system performance under **realistic client workloads**.
- Analyse the collected data to **identify the key performance determinants** within the system.

3.2 Methodology

This quantitative research is structured into four sequential stages:

3.2.1 Initial Scoping

The system consists of a central load balance server (main server) and a number of heterogeneous, distributed offload sites. In the real-world experiments, each system node runs within a containerised environment alongside network emulation tool to model realistic conditions.

The load balancing strategies will be deployed in both the real-world experiments and software simulations, coupled with a set of targeted optimisation features. For certain load balancing algorithms, a task duration prediction model is required.

The initial load balancing algorithms candidates include:

- Round Robin [SJA22]
- Interleaved Weighted Round Robin [TBB20]
- Equally Spread Current Execution [SJA22]
- Minimum Completion Time [BSB+01]
- Max-Min & Min-Min [SJA22, BSB+01]
- Ant Colony Optimisation [SJA22, LGS+23]

The initial computational optimizations include:

- Regular Segment and Asset Caching (LRU / LFU)
- Request Coalescing
- Proactive Request Prediction
- Partial Segment Caching with Greedy Dual Size Frequency (GDSF) policy [MQS+18]

The project will use generated request sequences across both the real-world experiment and the simulation. These sequences will be produced using an exponential probability density function for request interval and a Zipf function with an exponential cutoff for video selection [CKR+07].

Throughout the project, the following metrics will be used to evaluate the system performance:

- Server Utilisation Ratio (Execution Duty Cycle) $= \frac{\text{Active Execution Time}}{(\text{Active Execution Time} + \text{Idle Time})}$, paired with the execution pipeline specification.
- Client Perceived Latency (Average, P50, P90, P99)
- Caching cost (Memory or storage usage)

The following time measurements will be used to analyze internal system interactions and decompose end-to-end latency:

- T_{asset}: Time spent by the offload site retrieving and loading required media assets.
- T_{queuing}: Duration that a request waits in the offload site's scheduling queue.
- $T_{\text{work}} = T_{\text{decode}} + T_{\text{render}} + T_{\text{encode}}$: Active computational time on the offload site, decomposed into:
  - T_{decode}: Decompressing input video bitstreams.
  - T_{render}: Compositing layers, rasterizing overlays, and applying pixel transformations.
  - T_{encode}: Compressing the composited frames into the final output video segment.
- $T_{\text{offload-RTT}}$: Network round-trip time between the main server and the selected edge offload site + the load balancing decision overhead.
- $T_{\text{client-RTT}}$: Network round-trip time between the client playback device and the main server.

3.2.2 Real-World Experiment

In this stage, the prototype system is implemented and evaluated on a physical distributed infrastructure to gather formative performance data.

The software stack contains:

- Central Main Server: A containerised asynchronous Rust web server that handles public requests and make load balancing decisions.
- Offload Site Nodes: Heterogeneous containerised environments running a Rust proxy next to the Dana rendering engine with configurable execution pipelines (see Table 1).
- Network Infrastructure: A private point-to-point VPN (via Tailscale) interconnecting offload sites with the central server, alongside a cloud VM tunnel to expose public HTTPS endpoints.

To evaluate the load balancing algorithms and optimisation features under realistic conditions, the experiment introduces controlled environmental variables:

1. Hardware Heterogeneity: The offload site cluster specifications are controlled to construct both heterogeneous and homogeneous clusters across the distributed system.
2. Network Emulation: The Linux Traffic Control (`tc/netem`) utility is applied to create near and far offload clusters.
3. Request Sequences Skewness: The Zipf exponent α is substituted with 0.5 and 1.2 to produce a uniform sequence and a strongly concentrated sequence.

During each experiment, server side timing data is collected from HTTP response headers including T_{asset}, T_{queuing}, T_{decode}, T_{render}, T_{encode} and $T_{\text{offload-RTT}}$. The $T_{\text{client-RTT}}$ will be measured on the client side by recording $T_{\text{total}} - \text{Sum(server side timings)}$.

3.2.3 Software Simulation

While real-world experiments provide essential validation, they are naturally constrained in scale and cannot easily evaluate large clusters of heterogeneous offload nodes under sustained high-concurrency loads.

To overcome this scalability constraint, this stage develops a dedicated discrete-event simulator. This allows experiment to continue with large number of workload and offload sites, which create a significant scalability evaluation for load balancing algorithms and optimisation features. All event durations will be based on the formative data from the real-world experiments.

3.2.4 Analysis and Reflection

The final stage will analyse the findings from both physical experiments and simulation runs to determine key engineering conclusions:

- Cross Validation: The simulation outputs will be compared against the physical measurements to verify the simulator's accuracy and reliability.
- Bottleneck Analysis: The timing data will be analysed to identify common bottlenecks under various system conditions.
- Trade-Off Evaluation: The load balancing algorithms and optimisation features will be mapped across a Pareto frontier balancing three distinct objectives:
  1. Client-perceived latency: Maintain latency within a certain user abandonment threshold.
  2. Server utilisation: Maintain reasonable offload site computational workloads and avoid work underload or overload.
  3. Cache cost: Avoid excessive caching leading to memory or storage shortages.
- Design Recommendations: These recommendations will be based on the previous analysis results and provide promising directions for the OBM offload framework, focusing on load balancing algorithms and optimisation features.

4 Programme of Work

This project will run from early October 2026 until March 2027.

4.1 Timeline and Time Scale

Week 1 to 2: Initial Scoping (≈ 44 hours) (Section 3.2.1)
Week 3 to 7: Real-World Experiment (≈ 112 hours) (Section 3.2.2)
Week 8 to 12: Software Simulation (≈ 112 hours) (Section 3.2.3)
Week 13 to 15: Analysis and Reflection (≈ 66 hours) (Section 3.2.4)
Week 16 to 18: Final Report Writing (≈ 66 hours)

4.2 BCS Accreditation

Throughout the project, I will apply a number of computing, critical thinking and communication skills into the project.

During the project, I will apply communication skills from **SCC.200: Group Project** to coordinate with my supervisor.

For stage 1 Initial Scoping, I will apply the algorithmic knowledge from **SCC.121: Fundamentals of Computer Science** and critical thinking from **SCC.231: Computer Networks and Systems**.

For stage 2 Real-World Experiment and stage 3 Software Simulation, I will apply the software development principles from **SCC.111: Software Development & SCC.211: Software Design**, as well as the practical experience from **SCC.200: Group Project**.

For stage 4 Analysis and Reflection, I will apply the data analysis skills from **SCC.222: Artificial Intelligence Concepts & SCCM6220: Machine Learning**.

For stage 5 Final Report Writing, I will apply the writing skills from **SCC.141: Professionalism in Practice**.

References

[ABC+14] M. Armstrong, M. Brooks, A. Churnside, M. Evans, F. Melchior, and M. Shotton. *Object-based broadcasting - curation, responsiveness and user experience*, page 12.2. 2014.

[BSB+01] Tracy D Braun, Howard Jay Siegel, Noah Beck, Ladislau L Bölöni, Muthucumaru Maheswaran, Albert I Reuther, James P Robertson, Mitchell D Theys, Bin Yao, Debra Hensgen, and Richard F Freund. A comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems. *Journal of Parallel and Distributed Computing*, 61(6):810–837, 2001.

[CKR+07] Meeyoung Cha, Haewoon Kwak, Pablo Rodriguez, Yong-Yeol Ahn, and Sue Moon. I tube, you tube, everybody tubes: Analyzing the world's largest user generated content video system. In *Proceedings of the ACM SIGCOMM Internet Measurement Conference, IMC*, pages 1–14, 10 2007.

[DRP25] Paul Dean, Rajiv Ramdhany, and Barry Porter. Render cost driven caching for object-based media. In *2025 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*, pages 7–13, 2025.

[KS12] S. Shunmuga Krishnan and Ramesh K. Sitaraman. Video stream quality impacts viewer behavior: inferring causality using quasi-experimental designs. In *Proceedings of the 2012 Internet Measurement Conference*, IMC '12, page 211–224, New York, NY, USA, 2012. Association for Computing Machinery.

[LGS+23] Yogesh Lohumi, Durgaprasad Gangodkar, Prakash Srivastava, Mohammad Khan, Abdulrahman Alahmadi, and Ahmed Alahmadi. Load balancing in cloud environment: A state-of-the-art review. *IEEE Access*, 11:134517–134530, 12 2023.

[MQS+18] Tinghuai Ma, Jingjing Qu, Wenhai Shen, Yuan Tian, Abdullah Al-Dhelaan, and Mznah Al-Rodhaan. Weighted greedy dual size frequency based caching replacement algorithm. *IEEE Access*, 6:7214–7223, 2018.

[PRR26] Barry Porter, Rajiv Ramdhany, and Nicholas Race. Delivering layered object-based media using webassembly with selective cloud rendering. In *Proceedings of the ACM Multimedia Systems Conference 2026*, MMSys '26, page 238–249, New York, NY, USA, 2026. Association for Computing Machinery.

[SJA22] Dalia Abdulkareem Shafiq, N.Z. Jhanjhi, and Azween Abdullah. Load balancing techniques in cloud computing environment: A review. *Journal of King Saud University - Computer and Information Sciences*, 34(7):3910–3933, 2022.

[TBB20] Seyed Mohammadhossein Tabatabaee, Jean-Yves Le Boudec, and Marc Boyer. Interleaved weighted round-robin: A network calculus analysis. *CoRR*, abs/2003.08372, 2020.